



FIELD MANUAL

VERSION 1 · CARETAKER-COMPLIANT

Universal Governance Spec

Local Model Deployment



You are not adding governance to a system.
You are placing the system INSIDE governance.
All mutations pass through the kernel. No side paths. No trusted shortcuts.

COLOR SYSTEM

	BLUE	Architecture / structural sections
	RED	Failure modes / MUST NOTs / adversarial
	GOLD	Invariants / mandatory / final principles
	PURPLE	Advanced / extended / human-system

FRONT MATTER

Purpose & Operating Principle

Purpose

This manual defines ALL required knowledge, systems, and operational practices necessary to deploy the Universal Governance Execution Kernel on local AI systems. It is exhaustive, implementation-oriented, and strictly aligned with the locked specification.

Operating Principle

- You are not adding governance to a system.
- You are placing the system INSIDE governance.
- All mutations must pass through the kernel.
- There are no side paths, bypasses, or “trusted shortcuts.”

INVARIANT · Governance is the execution layer

The kernel is the only path through which state changes are allowed.

If anything can mutate state outside it, the system is invalid.

How to read this manual

Each numbered section opens its own page with a colored band. Blue marks architecture and structural sections. Red marks failure modes, prohibitions, and adversarial conditions. Gold marks invariants, mandatory rules, and final principles. Purple marks the extended and human-system layers (sections 12–50).

Verbatim specification text is preserved exactly as authored. Implementation-credibility expansions are appended inline as EXPANSION blocks demarcated with gold rules — they add worked examples, reference tables, and verification checklists without altering the locked spec.

§1

Foundational Knowledge Domains

You must understand the following disciplines at an implementation level:

1.1 Distributed Systems

- Deterministic processing
- Idempotent operations
- Event-driven architecture
- Failure isolation
- Replayability

1.2 Applied Cryptography

- SHA256 hashing (canonical JSON)
- Ed25519 signatures (sign/verify)
- Key management (generation, rotation, revocation)
- Integrity verification chains
- Non-repudiation

1.3 API Systems

- RESTful design (strict contracts)
- Schema validation (JSON schema enforcement)
- Request/response determinism
- Idempotency keys (traceld usage)

1.4 Data Persistence

- Append-only storage models
- SQLite / Postgres fundamentals
- Transaction integrity
- Write-ahead logging (WAL)
- Consistency guarantees

1.5 Policy Systems

- Rule engines (deterministic evaluation)
- Conflict resolution (deny-by-default)
- Policy scoping (resource, actor, operation)
- Risk-tier modeling

1.6 Identity & Authorization

- Capability-based security (not role-only)
- Token scoping and constraints
- Delegation attenuation
- Token introspection

1.7 Observability & Auditing

- Event sourcing
- Trace correlation (traceId propagation)
- Deterministic replay systems
- Audit integrity guarantees

1.8 Concurrency Control

- Race condition prevention
- Locking strategies (optimistic/pessimistic)
- Ordering guarantees for ledger writes
- Timestamp vs logical ordering

EXPANSION · Reference standards every implementer must cite

These are not suggestions; they are the authoritative source documents the kernel must conform to. Implementation reviews should verify each control against the cited standard.

Standard	Subject	Where it applies
FIPS 180-4	SHA-256 hashing	Hash chain integrity, payloadHash, recordHash
RFC 8032	Ed25519 EdDSA signatures	DecisionRecord, PER, approval signing
RFC 8785	JSON Canonicalization Scheme (JCS)	Canonical JSON for all hashed payloads
RFC 7519	JWT	CapabilityToken envelope (claim set, expiry)
NIST SP 800-207	Zero Trust Architecture	Capability-based access, no implicit trust
NIST SP 800-53	Security & Privacy Controls	AU-* audit, AC-* access, SC-* comm.
RFC 5905	NTPv4	Time synchronization for token / quorum expiry
RFC 6749 / 8252	OAuth 2.0 / Native Apps	Delegation attenuation patterns

§2

System Architecture Overview

All local deployments MUST implement the following components:

2.1 Governance API Layer

Endpoints (fixed, no additions):

```
POST /governance/evaluate
POST /governance/decisions
POST /capabilities/issue
POST /capabilities/introspect
POST /capabilities/revoke
POST /observability/events
GET  /observability/traces/{traceId}
POST /observability/traces/{traceId}/replay
```

2.2 Enforcement Kernel

Implements:

- Invariants
- Evaluation pipeline
- Quorum engine
- Decision synthesis

2.3 DecisionRecord Ledger

- Append-only
- Hash-linked
- Persisted (SQLite/Postgres)
- Rejects chain breaks

2.4 CapabilityToken Authority

- Token issuance
- Token validation
- Revocation handling
- Delegation enforcement

2.5 Quorum Engine

- Approval aggregation
- Signature validation
- Threshold enforcement
- Governance evaluation per approval

2.6 Observability System

- Event emission
- Immutable payload storage

- Trace reconstruction
- Replay engine

2.7 Compensation Engine

- Forward-only corrective mutations
- Binding to original decision
- Policy + quorum governed

§2 · CONT.

Endpoint Contract (Canonical Reference)

Each endpoint enforces a strict request/response schema. Idempotency is keyed on traceId; duplicate submissions return the original DecisionRecord, never a divergent one.

Verb	Path	Request	Response	Idem. key
POST	/governance/evaluate	PER	PolicyEvaluationResult	traceId
POST	/governance/decisions	DecisionRecordSubmit	DecisionRecord	traceId
POST	/capabilities/issue	TokenIssueRequest	CapabilityToken	requestId
POST	/capabilities/introspect	{token}	TokenIntrospection	—
POST	/capabilities/revoke	RevokeRequest	RevokeReceipt	requestId
POST	/observability/events	ObservabilityEvent	Ack(eventId)	eventId
GET	/observability/traces/{traceId}	—	TraceBundle	—
POST	/observability/traces/{traceId}/replay	ReplayReq	ReplayResult	replayId

§3

Governance Execution Flow (Mandatory)

Every mutation MUST follow EXACTLY:

Step 1 — Construct PER

- Include actor, action, resource, mutation, traceId
- Compute inputHash (canonical JSON)

Step 2 — Call `/governance/evaluate`

- Submit PER
- Receive PolicyEvaluationResult

Step 3 — Evaluate Outcome

- allow → proceed
- deny → stop
- escalate → wait for quorum

Step 4 — If quorum required

- Collect QuorumApproval(s)
- Each approval must:
 - Have capabilityToken
 - Be governance-evaluated
 - Be signed (ed25519)

Step 5 — Write DecisionRecord

- Include:
 - inputHash
 - outputHash
 - quorum data
- Compute recordHash (canonical JSON)

Step 6 — Emit Observability Event

- payloadHash must match canonical payload
- payloadRef must be immutable

Step 7 — Create MutationGovernanceBinding

- Bind token + evaluation + decision + quorum

Step 8 — Execute Mutation

- Only after binding verification

MANDATORY · No step may be skipped or reordered.

A binding that omits any predecessor is invalid by definition. The kernel rejects it before the mutation reaches the resource adapter.

§3 · CONT.

Worked Example — Governed Mutation, End-to-End

A local model proposes resource.user.email change. The proposal is wrapped, evaluated, quorum-approved, recorded, and only then committed:

- 1) PER (canonical, sorted keys):

```
{
  "action": "update",
  "actor": {
    "id": "agent:llm-7",
    "type": "model"
  },
  "mutation": {
    "after": {
      "email": "a@b.io"
    },
    "before": {
      "email": "a@b.com"
    }
  },
  "resource": {
    "classification": "restricted",
    "id": "user:42"
  },
  "schemaVersion": "1",
  "traceId": "01HX...QM"
}
```

- 2) POST /governance/evaluate -> PEResult{outcome:escalate, requiredQuorum:2}
- 3) Collect 2 QuorumApproval rows, each ed25519-signed by approver key, each with valid CapabilityToken (scope=approve:user.update, riskTier<=restricted).
- 4) DecisionRecord:

```
{
  "inputHash": "...",
  "outputHash": "...",
  "outcome": "allow",
  "prevRecordHash": "<H_n-1>",
  "quorum": {
    "aggregatedHash": "...",
    "approvals": [...]
  },
  "schemaVersion": "1",
  "timestamp": "2026-04-27T10:08:00Z",
  "traceId": "..."
}
```

recordHash = sha256(canonical_json(DecisionRecord))
- 5) ObservabilityEvent emitted with payloadHash matching canonical payload.
- 6) MutationGovernanceBinding created:
token + PEResult + DecisionRecord + quorum.
- 7) Adapter writes the new email ONLY after binding verification succeeds.

§4

Data Integrity Requirements

4.1 Canonical JSON

- All hashes use canonical JSON
- No field ordering variation
- No omitted fields

4.2 Hash Chain

- Each DecisionRecord links to previous
- Chain break = system halt

4.3 Determinism

- Same input MUST produce same output
- Required for replay validation

EXPANSION · Canonical JSON rules (RFC 8785 / JCS-aligned)

Implementations should treat canonicalization as a single-purpose, well-tested module with no per-call options. Drift here breaks every hash downstream.

- Sort all object keys lexicographically (codepoint order).
- Encode UTF-8; no BOM; escape only required JSON control characters.
- Numbers: shortest round-trippable form; no trailing zeros; no exponent unless required.
- Booleans / null: lowercase literals only.
- Arrays: preserve order (semantic); never sort.
- No whitespace anywhere; no comments; no duplicate keys.

```
# Reference implementation contract (Python)
import json, hashlib
def canonical(obj):
    return json.dumps(obj, sort_keys=True, separators=(',', ':'),
                      ensure_ascii=False, allow_nan=False).encode('utf-8')
def sha256_canon(obj):
    return hashlib.sha256(canonical(obj)).hexdigest()
```

§5

Capability Token System

You must implement:

- Issuance (signed by governanceAuthority)
- Introspection (runtime validation)
- Revocation (immediate effect)

Validation MUST check:

- Signature validity
- Expiry
- Scope (operation/resource)
- Constraints (riskTier, jurisdiction)

MANDATORY · No token = no mutation.

Every governed mutation must reference a valid, introspected, non-revoked CapabilityToken bound to the actor and scoped to the operation.

EXPANSION · CapabilityToken envelope (reference)

```
{
  "schemaVersion": "1",
  "tokenId": "tok_01HX...",
  "issuer": "governanceAuthority/v3",
  "subject": {"id": "agent:llm-7", "type": "model"},
  "scope": ["update:user.email", "read:user.*"],
  "constraints": {
    "riskTier": "restricted",
    "jurisdiction": "us-only",
    "rateLimit": "60/min"
  },
  "delegation": {"depthMax": 2, "attenuationOnly": true},
  "issuedAt": "2026-04-27T10:08:00Z",
  "notBefore": "2026-04-27T10:08:00Z",
  "expiresAt": "2026-04-27T10:38:00Z",
  "kid": "auth-key-2026-04",
  "sig": "<ed25519 over canonical body w/o sig>"
}
```

§6

Policy Engine

Policy engine MUST:

- Evaluate PER against policyRefs
- Enforce deny-by-default
- Resolve conflicts deterministically
- Output structured decision basis

Policies control:

- Allowed operations
- Required quorum
- Risk tier
- Resource access

EXPANSION · Risk tier × classification × quorum mapping (reference)

This table is the canonical mapping the policy engine must enforce. Adjust thresholds in the policy version, not in code.

Classification	Mutation kind	Risk tier	Min approvals	Quorum rule
public	softMutation	low	0	single signer, single approval
internal	softMutation	low	0	single signer
internal	hardMutation	medium	1	1 approver, role:Operator
restricted	hardMutation	high	2	2 approvers, distinct roles
restricted	governanceMutation	high	2	2 approvers, must include PolicyAuthority
critical	any	critical	3	3 approvers, distinct roles, <15 min window

§7

Quorum System

Required for governanceMutation (and some hardMutation):

You must support:

- Role-based approval validation
- CapabilityToken per approver
- Governance evaluation per approval
- Signature verification
- Deduplication (first valid approval wins)

Failure conditions:

- Expired quorum window
- Invalid signatures
- Insufficient approvals

§8

DecisionRecord Ledger

Requirements:

- Append-only
- Hash-linked
- Durable storage
- Cryptographically signed

Must include:

- inputHash
- outputHash
- prevRecordHash
- quorum.aggregatedHash
- recordHash

INVARIANT · Ledger is the source of truth.

Every other store is derived. If a derived store disagrees with the ledger, the derived store is wrong; never the other way around.

EXPANSION · DecisionRecord schema (reference)

```
{
  "schemaVersion": "1",
  "recordId": "drec_01HX...",
  "traceId": "01HX...",
  "inputHash": "<sha256 of PER>",
  "outputHash": "<sha256 of canonical decision body>",
  "outcome": "allow|deny|escalate|compensate",
  "policyRefs": ["policy:user.update@v3"],
  "quorum": {
    "approvals": [ {"approverId":"...", "tokenId":"...", "sig":"<ed25519>"} ],
    "aggregatedHash": "<sha256 of canonical approvals[]>"
  },
  "prevRecordHash": "<sha256 of prior record canonical body>",
  "timestamp": "RFC3339 UTC, monotonic",
  "authoritySig": "<ed25519 over canonical body w/o this field>",
  "recordHash": "<sha256 of canonical body INCLUDING authoritySig>"
}
```

§9

Observability & Replay

You must implement:

9.1 Event Emission

- Every step emits event
- Must include traceId

9.2 Immutable Storage

- payloadRef points to immutable storage
- payloadHash validates content

9.3 Replay Engine

Modes:

- verify → must match ledger
- simulate → no writes

Replay MUST:

- Recompute hashes
- Detect mismatches
- Emit error events on divergence

§10

Compensation System

INVARIANT · Compensation is NOT rollback.

It is a NEW mutation, forward-only, linked to the original decision.

It is:

- A NEW mutation
- Forward-only
- Linked to original decision

Requirements:

- Must pass governance evaluation
- Must reference `originalDecisionRecordId`
- Must include `originalRecordHash`
- Must produce `DecisionRecord(outcome=compensate)`

§11

Security Model

You must enforce:

- Zero trust (no implicit trust)
- Capability-based access
- Signed approvals
- Signed decisions
- Immutable logs

Attack surfaces to protect:

- Token forgery
- Approval spoofing
- Ledger tampering
- Replay manipulation

LOCAL MODELS

Local Model Integration

For local LLMs (GPT, LLaMA, Mistral, etc.):

ZERO TRUST · The model is NOT trusted.

It is treated as `subject.actorType = model`. The model generates intent and DOES NOT execute mutations.

All outputs must be:

- Wrapped into PER
- Evaluated before execution

FAILURE

Failure Handling

You must handle:

- Evaluation failure → deny
- Token invalid → deny
- Quorum timeout → deny
- Ledger inconsistency → halt system
- Replay mismatch → raise error event

FAIL-CLOSED · System must fail CLOSED.

No partial execution. No retry-until-success. The denial itself is recorded as a DecisionRecord and emitted to observability.

EXPANSION · Failure → Detection → Response matrix

Failure	Detection	Response
Token forgery	Signature verify fail	deny + revoke + alert security
Token replay	tokenId already bound	deny + record violation
Approval spoof	Approver signature fail	deny + invalidate quorum window
Ledger chain break	prevRecordHash mismatch	halt all mutations + page on-call
Replay divergence	Recomputed hash differs	emit error event + auditor flag
Storage failure	Write or fsync error	deny; queueing is forbidden
Clock drift	ts > leeway (NTP fail)	deny + force re-sync + alert
Quorum timeout	window expired	deny + DecisionRecord(outcome=deny)

PERFORMANCE

Performance Considerations

You should design for:

- Low-latency evaluation
- Efficient hashing
- Indexed ledger queries
- Parallel quorum collection

But NEVER at the cost of:

- determinism
- auditability
- correctness

DEPLOY

Deployment Checklist

Before going live:

☐	Governance endpoints operational
☐	Ledger persistence verified
☐	Hash chain integrity tested
☐	Replay engine validated
☐	Token authority initialized
☐	Quorum policies configured
☐	Observability pipeline active

EXPANSION · Pre-flight verification (signable)

Each row should be signed-off by a named operator and dated. Keep this page in the runbook.

☐	Governance Authority root keypair generated; public key distributed; private key in HSM/TPM
☐	Initial CapabilityToken issued to Operator role; introspection live
☐	Policy set v1 loaded; deny-by-default verified against negative tests
☐	Genesis DecisionRecord written: prevRecordHash = 0, signed, recordHash captured
☐	Append-only ledger storage (WAL on, fsync per record) confirmed
☐	Observability pipeline emits + stores immutable payloadRef; payloadHash check passes
☐	Replay engine verify-mode against last 1,000 records: zero divergence
☐	NTP time source synced; max drift < 250ms across nodes
☐	Quorum thresholds reflect risk-tier × classification matrix
☐	Resource adapters reject any direct write that lacks MutationGovernanceBinding
☐	Backup target tested with hash-chain integrity verification on restore
☐	Chaos test: token revocation mid-flight → mutation correctly denied

PROHIBITIONS

What You MUST NOT Do

- Do NOT bypass /governance/evaluate
- Do NOT mutate without DecisionRecord
- Do NOT allow unsigned approvals
- Do NOT modify ledger entries
- Do NOT introduce new primitives

PROHIBITION · These are violations, not warnings.

Any of the above is a system invariant breach and must trigger immediate halt + escalation. The system is invalid until reverified.

MENTAL MODEL

Mental Model

Think of the system as:

- A blockchain-like decision ledger
- A policy firewall for all mutations
- A cryptographic audit system
- A deterministic execution engine

FINAL STATE

Final State (Core)

If implemented correctly:

- Every action is governed
- Every decision is provable
- Every mutation is reversible (via compensation)
- Every failure is detectable
- Every system is auditable

END OF FIELD MANUAL

EXTENDED

Additional Required Knowledge — Field Manual Supplement

This section defines advanced, non-optional operational knowledge required to successfully deploy, secure, and sustain the Universal Governance Spec (v1) in real-world local-model environments.

These are NOT new primitives. These are REQUIRED operational disciplines derived from the governing system.

Each section below opens its own page. The cumulative requirement is that all sections 12–32 (operational layer) and 33–50 (human + system layer) are implemented alongside the core spec for a deployment to qualify as caretaker-compliant.

§12

Cryptographic Key Management (Mandatory)

Governance Authority Keys

- ed25519 keypair REQUIRED for:
 - DecisionRecord signing
 - PolicyEvaluationResult signing
 - CapabilityToken issuance
- Private keys MUST:
 - Never leave secure boundary (HSM, TPM, enclave, or equivalent)
 - Be non-exportable when possible
- Public keys MUST:
 - Be distributed to all verification nodes
 - Be versioned and rotated

Key Rotation

- MUST support rotation without breaking verification
- Old keys MUST remain valid for historical verification
- Rotation events MUST emit ObservabilityEvent(type=capability.issued or equivalent)

Compromise Handling

- Immediate revocation
- Emit observability event
- Invalidate affected tokens
- Flag replay verification anomalies

EXPANSION · Key lifecycle reference

Stage	Mechanism	Operational note
Generate	HSM / TPM / sealed enclave	FIPS 140-2 L3+ where available
Distribute (pub)	Signed key bundle, multi-channel	Pin in adapters; version=N
Use	Sign DecisionRecord, PER, tokens	kid claim required
Rotate	Issue v=N+1, accept N and N+1	Emit capability.issued event
Retire	Stop signing with N; verify only	Keep N for historical verification
Revoke	Compromise: invalidate N + tokens	Cascade: re-verify recent records

§13

Canonical JSON Standardization (Critical)

All hashing depends on canonical JSON.

Canonicalization MUST:

- Sort keys lexicographically
- Remove whitespace variability
- Enforce consistent encoding (UTF-8)
- Normalize numbers, booleans, nulls

Failure results in:

- Hash mismatch
- Replay failure
- Ledger invalidation

CRITICAL · This is a system-critical dependency.

See §4.1 expansion for the reference implementation contract. Treat the canonicalizer as fixed; never 'optimize' per call site.

§14

Time Synchronization (Strict Requirement)

All timestamps affect:

- quorum expiry
- token expiry
- replay determinism

System MUST:

- Use synchronized time source (e.g., NTP with fallback)
- Enforce monotonic progression where required

Clock drift risks:

- false quorum failure
- token misvalidation
- replay divergence

EXPANSION · Time discipline thresholds

Recommended budgets — deployments may tighten, never loosen, without justification.

Discipline	Budget	Action on breach
Inter-node clock skew	< 250 ms	Halt writes if exceeded
Token expiry leeway	< 5 s	Deny on violation
Quorum window default	5–15 min	Per risk tier; critical < 15 min
Replay timestamp tolerance	Exact match	Use logical ordering for ties

§15

State Hashing Strategy (Mandatory)

Every governed resource **MUST** support deterministic state hashing.

Requirements:

- Same state → same hash
- No nondeterministic fields (timestamps, random IDs unless excluded)
- Hash boundary **MUST** be clearly defined

Used for:

- beforeHash validation
- afterHash verification
- compensation correctness

§16

Resource Adapter Layer (Required for Local Models)

Local systems **MUST** implement adapters that:

Translate:

- model actions → proposedMutation
- system state → beforeHash/afterHash

Enforce:

- no direct mutation bypass
- all writes go through governance pipeline

ARCHITECTURE · Adapters are enforcement chokepoints.

If a write path exists outside the adapter, it is the bypass. Inventory every storage write and confirm each one ends at an adapter call.

§17

Failure Modes (Fail-Closed Requirement)

System MUST default to DENY on:

- token introspection failure
- policy evaluation failure
- quorum uncertainty
- hash mismatch
- storage failure
- observability failure

FAIL-CLOSED · No partial execution allowed.

A 'mostly-applied' mutation is worse than a denied one. The kernel never half-commits.

§18

Idempotency + Retry Safety

All governance operations **MUST** be idempotent:

- Re-submitting same PER **MUST NOT** produce divergent outcome
- DecisionRecord must not duplicate inconsistently
- Quorum approvals must respect deduplication rules

Use:

- requestId
- evaluationId
- approvalId

to enforce uniqueness.

§19

Concurrency Control

Parallel mutations MUST:

- not violate hash chain integrity
- not overwrite shared state without detection

Mechanisms:

- optimistic locking (hash comparison)
- or serialized mutation execution

Conflict outcome:

- → DENY + DecisionRecord

§20

Storage Hardening

Ledger + Observability storage MUST ensure:

- append-only guarantees
- tamper resistance
- crash consistency

Recommended:

- WAL (write-ahead logging)
- checksums
- replication
- periodic integrity verification

Optional:

- Merkle tree anchoring
- external notarization

§21

Trace Propagation Discipline

traceld MUST:

- originate at entry point
- propagate through:
 - PER
 - PEResult
 - DecisionRecord
 - QuorumApproval
 - ObservabilityEvent

Breakage = audit failure.

§22

Policy Engine Requirements

Policy system MUST support:

- deterministic evaluation
- conflict resolution (denyOnConflict)
- versioning
- traceable policyRefs

Policies MUST NOT:

- override invariants
- bypass quorum rules
- weaken capability enforcement

§23

Capability Token Enforcement (Deep)

Validation MUST include:

- signature verification
- expiry check
- status check (revoked/suspended)
- scope validation
- constraint validation

Delegation:

- MUST be attenuation-only
- MUST enforce depth limits

§24

Quorum Security Hardening

Approvals MUST:

- be individually signed
- be independently validated
- require valid capability token

System MUST prevent:

- replay of approvals
- duplicate approvals
- forged roles

§25

Compensation Safety Controls

Compensation MUST:

- verify current state via beforeHash
- never assume state correctness
- never cascade uncontrolled

Optional control:

- limit number of compensations per original record

§26

Replay System Validation

Replay MUST:

- recompute ALL hashes
- re-evaluate ALL policies (in verify mode)
- detect ANY divergence

Replay mismatch MUST:

- emit error event
- flag integrity violation

§27

Deployment Architecture (Local Model Environment)

Minimum components:

- Governance API layer (/governance/evaluate)
- Policy engine
- Token authority
- Ledger storage
- Observability storage
- Resource adapters

Optional separation:

- control plane vs execution plane

§28

Security Boundaries

Define strict boundaries between:

- model execution
- governance enforcement
- storage layers
- key management

No model may:

- self-issue tokens
- self-approve quorum
- bypass evaluation

§29

Testing Requirements

Must include:

- invariant violation tests
- replay determinism tests
- quorum edge cases
- token revocation scenarios
- compensation correctness

System is not valid without passing these.

§30

Operational Monitoring

Monitor:

- decision throughput
- denial rates
- quorum delays
- replay failures
- hash mismatches

Alert on:

- replay mismatch
- invalid signature
- ledger break attempt

§31

Backup + Recovery

Backups MUST:

- preserve hash chain integrity
- include ledger + observability

Recovery MUST:

- verify hashes before restore acceptance

§32

Zero-Trust Assumption

System MUST assume:

- every component can fail or be compromised
- verification is mandatory at every step

Trust is cryptographic, not positional.

MID-MANUAL

Final Condition (Operational Layer)

This knowledge is REQUIRED to make the Universal Governance Spec:

- actually enforceable
- actually secure
- actually deterministic
- actually auditable

Without these operational disciplines:

The specification remains correct, but the system becomes vulnerable, non-deterministic, or unverifiable.

This completes the field manual knowledge layer.

FINAL SUPPLEMENT

Real-World Execution + Human System Integration

Short answer: yes — there is still one final layer most people miss, and without it, real-world deployments fail even if everything else is correct.

Below is the last complete supplement. After this, you are truly at “nothing else missing” for a field manual.

This layer covers the gap between a perfectly designed governance system and a system that actually survives real-world use, operators, and adversarial conditions.

§33

Human Governance Model (Non-Optional)

You must define:

- Who can issue CapabilityTokens
- Who can approve quorum
- Who defines policy
- Who can trigger compensation
- Who audits the system

Required roles (minimum):

- GovernanceAuthority (root signer)
- PolicyAuthority (defines policies)
- QuorumApprover(s) (multi-party)
- Auditor (read-only, replay authority)
- Operator (runs system, no override power)

RULE · Separation of duties

No single role should be able to: issue token + approve quorum + define policy. That is a critical separation-of-duties control.

EXPANSION · Role × Capability matrix (least-privilege reference)

Role	Issue token	Define policy	Approve quorum	Audit/replay	Sign decisions
GovernanceAuthority	✓	—	—	—	✓ (sign root)
PolicyAuthority	—	✓	—	—	—
QuorumApprover	—	—	✓	—	—
Auditor	—	—	—	✓ replay/verify	—
Operator	—	—	—	—	✓ run only

§34

Bootstrap / Genesis State (Critical)

You must define how the system starts.

Required:

- First GovernanceAuthority keypair (root of trust)
- First CapabilityToken issuance (who gets initial authority)
- First DecisionRecord (genesis record with prevRecordHash = 0)
- Initial policy set

Without a clean genesis:

- the ledger cannot be trusted
- replay cannot be verified

§35

Policy Lifecycle Management

Policies are the ONLY allowed extension surface.

You must support:

- versioning (policy v1, v2, etc.)
- activation timestamps
- deprecation (but not deletion)
- traceability (policyRefs MUST map to versioned policy)

RULE · No retroactive policy changes

Policy changes MUST NOT retroactively alter past decisions. Replay uses current policy in verify mode (drift detection) and historical ledger as truth.

§36

Schema Versioning Strategy

All primitives already have schemaVersion.

You must define:

- how upgrades happen
- how backward compatibility is preserved
- how replay handles mixed versions

RULE · Old DecisionRecords MUST remain valid forever.

No breaking changes allowed.

§37

Adversarial Model Assumption

You must assume:

- malicious actors with valid tokens
- compromised services
- replay attacks
- forged inputs
- timing attacks

Controls already defined must be enforced strictly:

- signature validation
- capability scope enforcement
- deduplication
- hash verification

Never trust:

- inputs
- models
- operators

Only trust:

- cryptographic verification

§38

Data Classification + Policy Mapping

Your PER includes resource.classification.

You must define:

- what “public / internal / restricted / critical” means
- how policies differ per classification
- how riskTier is derived from classification

Example (must be defined in your system):

- critical → always quorum
- restricted → policy-dependent quorum
- public → no quorum

Without this:

- policy engine cannot function correctly

EXPANSION · Classification → derived risk tier (reference)

Classification	Definition (example)	Risk tier	Allowed mutation kinds
public	Public-facing data, no PII	low	softMutation only
internal	Org-only, non-sensitive	medium	soft + hard with quorum=1
restricted	PII, secrets, governance state	high	hard + governance, quorum≥2
critical	Root keys, ledger, genesis, kernel config	critical	always quorum, distinct roles, <15 min window

§39

Latency + Performance Tradeoffs

Governance adds latency.

You must design for:

- synchronous vs async quorum
- batching where allowed (never skip evaluation)
- caching token introspection (within validity window)

RULE · Performance optimizations MUST NOT bypass:

evaluation

decision recording

observability

§40

Offline / Degraded Mode Strategy

Define behavior when:

- governance service unavailable
- token introspection fails
- storage unavailable

Required behavior:

FAIL-CLOSED · No mutation allowed.

Optional: queue requests for later evaluation (but do not execute).

§41

Multi-Node Consistency

If distributed:

- all nodes **MUST** agree on ledger order
- prevRecordHash must be globally consistent

Use:

- leader-based write
- or consensus mechanism (Raft/Paxos-style)

Ledger divergence = system failure.

§42

Integration with Local Models

Critical rule:

RULE · Models are UNTRUSTED.

They propose mutations and NEVER execute mutations directly. All model outputs MUST be translated into PER. No exception.

§43

Prompt / Model Hardening

Even though governance enforces execution:

You must still:

- constrain prompts
- prevent instruction injection
- sanitize outputs

Reason:

Governance stops bad execution, but not bad proposals flooding the system.

§44

Rate Limiting + Abuse Control

Prevent:

- PER flooding
- quorum spam
- replay abuse

Controls:

- per-actor rate limits
- per-token limits
- anomaly detection

§45

Legal + Compliance Mapping (Optional but Practical)

Map system to:

- SOC2 (audit trail → DecisionRecord + Observability)
- GDPR (traceability + controlled mutation)
- NIST (zero trust + verification)

Not required for function, but required for real-world adoption.

EXPANSION · Compliance crosswalk

Standard	Requirement	How this kernel satisfies it
SOC2 CC7.2	System monitoring & anomalies	Observability events + alerts on §30
SOC2 CC8.1	Change management	DecisionRecord per mutation; policy versioning §35
GDPR Art. 30	Records of processing	Append-only ledger + traceId propagation §21
GDPR Art. 17	Right to erasure	Compensation §10 (forward-only erasure record)
NIST 800-207	Zero Trust Architecture	Capability tokens §5/§23, no implicit trust §32
NIST 800-53 AU-2/AU-9	Audit events / protection	Immutable observability storage §9.2 / §20
ISO 27001 A.12.4	Logging & monitoring	Hash-linked ledger + replay verification §26

§46

Documentation + Operator Training

Operators MUST understand:

- invariants cannot be bypassed
- compensation is forward-only
- denial is correct behavior

Most system failures come from:

- humans trying to “override” governance

That must be impossible.

§47

Chaos Testing (Highly Recommended)

Simulate:

- token revocation mid-flight
- quorum timeout
- hash mismatch
- storage failure
- replay mismatch

Verify:

- system always fails safely

EXPANSION · [Chaos test catalog](#)

ID	Scenario	Expected behavior
CT-01	Revoke token between evaluate and binding	deny + DecisionRecord(deny)
CT-02	Quorum approver offline; window expires	deny + DecisionRecord(deny, reason=quorum_timeout)
CT-03	Mutate prevRecordHash on disk	halt + alert; chain break detected on next read
CT-04	Inject duplicate approval (same approverId)	dedup; only first valid wins
CT-05	Storage fsync error during DecisionRecord write	deny; no partial commit; observability alert
CT-06	Replay verify on tampered payloadRef	error event emitted; integrity violation flag
CT-07	Clock skew node B by +90s	deny token-expiry edge cases; NTP alert
CT-08	Forge approval signature with wrong key	verify fail; deny; alert security

§48

System Boundary Definition

You must explicitly define:

What IS governed:

- all state mutations
- all model actions affecting state

What is NOT governed:

- read-only operations
- external systems (unless integrated)

Ambiguity here creates bypass risk.

§49

Upgrade / Migration Strategy

When deploying new versions:

- freeze writes if necessary
- verify ledger integrity pre/post
- maintain replay compatibility

Never:

- rewrite ledger
- alter historical hashes

§50

Final Principle — Governance Is the Execution Layer

This is the most important conceptual requirement:

The system you built is NOT:

- a logging system
- a policy checker
- an approval workflow

It IS:

INVARIANT · the ONLY path through which state changes are allowed

If anything can mutate state outside it, the system is invalid.

FINAL STATE

If you implement all three layers

If you implement:

- Core Spec (v1 LOCKED)
- Operational Layer (12–32)
- Human + System Layer (33–50)

Then you have:

- A fully enforceable
- fully auditable
- fully deterministic
- fully adversarially resilient
- governance execution system

CLOSING · There is nothing missing after this.

Anything further is optimization, scaling, or domain-specific policy — not foundational knowledge.

APPENDIX A

Glossary

Term	Definition
PER	Proposed Execution Request — actor + action + resource + mutation + traceId; the input to /governance/evaluate.
PEResult	PolicyEvaluationResult — outcome (allow/deny/escalate), policyRefs, basis.
DecisionRecord	Append-only, hash-linked, signed record of an evaluated mutation.
MGB	MutationGovernanceBinding — token + PEResult + DecisionRecord + quorum, required pre-execution.
CapabilityToken	Signed, scoped, attenuable authorization. No token = no mutation.
QuorumApproval	Individually signed, governance-evaluated approval bound to a token.
traceId	End-to-end correlation id; mandatory across all primitives.
Compensation	Forward-only corrective mutation referencing originalDecisionRecordId.
payloadHash / payloadRef	Content hash + immutable storage pointer for observability events.
recordHash	sha256 of canonical DecisionRecord including authoritySig.
prevRecordHash	Link to the prior DecisionRecord; chain break = halt.
Genesis record	First DecisionRecord; prevRecordHash = 0; root of the ledger.

APPENDIX B

Primary References

- FIPS PUB 180-4, Secure Hash Standard (SHS), NIST.
<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>
- RFC 8032, Edwards-Curve Digital Signature Algorithm (EdDSA).
<https://www.rfc-editor.org/rfc/rfc8032>
- RFC 8785, JSON Canonicalization Scheme (JCS). <https://www.rfc-editor.org/rfc/rfc8785>
- RFC 7519, JSON Web Token (JWT). <https://www.rfc-editor.org/rfc/rfc7519>
- NIST SP 800-207, Zero Trust Architecture.
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf>
- NIST SP 800-53 Rev. 5, Security and Privacy Controls.
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r5.pdf>
- RFC 5905, Network Time Protocol Version 4. <https://www.rfc-editor.org/rfc/rfc5905>
- RFC 6749, The OAuth 2.0 Authorization Framework. <https://www.rfc-editor.org/rfc/rfc6749>
- AICPA, SOC 2 Trust Services Criteria (TSC). aicpa-cima.com
- EU GDPR, Regulation (EU) 2016/679. eur-lex.europa.eu/eli/reg/2016/679/oj

FIELD MANUAL · UNIVERSAL GOVERNANCE SPEC v1 · CARETAKER-COMPLIANT · END